

Aula 4 — Maestro Cross-Platform & Cloud Devices

"Maestro vale a pena? Appium morreu?"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 22/06/2026

Recap da Aula 3

- **Setup do projeto** RN com Jest + RNTL configurados
- **Domain puro:** `posterUrl`, `isTokenError`, `favoritesStore` com Jest
- **Componentes RNTL:** `MovieCard` — render, toque, navegação mockada
- **Integração:** `NavigationContainer`, `QueryClientProvider`, `renderHook`
- Diferença unit × integração × E2E na pirâmide

Atividade 2 (suíte unitária, até 21/06) — entregue?

Agenda da aula (3h30)

1. **Bloco 1 (90min)** — Maestro deep dive + comparativo Appium
2. **Intervalo (15min)**
3. **Bloco 2 (90min)** — Hands-on: suíte Maestro completa + Firebase Test Lab
4. **Esquenta (15min)** — Atividade 3

Objetivos de aprendizagem

- **Implementar** suítes Maestro com YAML declarativo
- **Aplicar** conditional flows e fragmentos reutilizáveis
- **Comparar** Maestro vs Appium 2 com matriz quantitativa
- **Executar** flows em Firebase Test Lab e BrowserStack
- **Aplicar** estratégia top 10 devices (StatCounter)

Por que Maestro venceu (em adoção real)

Reddit, Cash App, Headspace, Loop Habit Tracker — adoção crescente desde 2022.

Razões técnicas:

1. **YAML declarativo** vs código imperativo
2. **Retry built-in** elimina sleeps
3. **Sem WebDriver Protocol** — fala direto com device
4. **Setup minutos**, não dias
5. **Cross-platform real** — mesmo flow iOS + Android
6. **Free** + cloud opcional

! Não é "killer" do Appium. É alternativa madura para o caso comum.

Maestro — primeiro flow

```
# login_flow.yaml
appId: com.myapp
---
- launchApp
- tapOn: "Email"
- inputText: "user@email.com"
- tapOn: "Senha"
- inputText: "secret"
- tapOn: "Entrar"
- assertVisible: "Bem-vindo"
```

```
maestro test login_flow.yaml
```

Em 30 segundos você tem teste E2E rodando.

Maestro — comandos principais

```
- launchApp                # abre app
- tapOn: "Submit"          # tap por texto
- tapOn:
  id: "submit-button"      # tap por ID
- inputText: "hello"       # digita
- swipe:                   # gesto
  start: "50%, 80%"
  end: "50%, 20%"
- scroll                    # scroll padrão
- assertVisible: "Welcome" # asserção
- assertNotVisible: "Loading"
- pressKey: "BACK"         # botão sistema
- waitForAnimationToEnd    # sincronização explícita
- takeScreenshot: "after-login"
```

Maestro — conditional flows

```
appId: com.myapp
---
- launchApp
- runFlow:
  when:
    visible: "Aceitar cookies"
    file: ./flows/dismiss-cookies.yaml
- runFlow:
  when:
    notVisible: "Welcome"
    file: ./flows/login.yaml
- assertVisible: "Welcome"
```

Suíte se adapta ao estado do app. Sem `if` em código imperativo.

Maestro — fragmentos reutilizáveis (`runFlow`)

```
# flows/login.yaml (fragmento)
- tapOn: "Email"
- inputText: "user@email.com"
- tapOn: "Senha"
- inputText: "secret"
- tapOn: "Entrar"
```

```
# tests/checkout.yaml (usa fragmento)
appId: com.myapp
---
- launchApp
- runFlow: ../flows/login.yaml
- tapOn: "Carrinho"
- tapOn: "Comprar"
- assertVisible: "Pedido confirmado"
```

DRY em testes E2E.

Maestro — parametrização via env

```
# tests/login.yaml
appId: com.myapp
env:
  EMAIL: "default@email.com"
  PASSWORD: "secret"
---
- launchApp
- inputText: ${EMAIL}
- inputText: ${PASSWORD}
```

```
maestro test login.yaml -e EMAIL=admin@myapp.com -e PASSWORD=adminPass
```

Reutiliza flow para múltiplas personas.

Maestro — JavaScript scripts

```
appId: com.myapp
---
- runScript: |
    output.fakeEmail = `user-${Date.now()}@test.com`;
    output.fakePassword = Math.random().toString(36).substring(2, 12);
- launchApp
- inputText: ${output.fakeEmail}
- inputText: ${output.fakePassword}
- tapOn: "Cadastrar"
```

Para casos onde YAML não basta (geração de dado, lógica condicional complexa).

Maestro Studio — record & replay

```
maestro studio
```

Abre interface:

- Mostra hierarquia do app em tempo real
- Permite tap visual e gera comando YAML
- Replay com inspeção
- Mais didático para aprender que documentação

Ferramenta de produtividade real, especialmente para iniciantes.

Maestro Cloud

Execução remota em devices reais.

```
maestro cloud --apiKey=$MAESTRO_API_KEY \  
  --app-file=app.apk \  
  flows/
```

- Reports HTML com screenshots e vídeo
- Devices reais
- Pago após free tier inicial

Alternativa: integrar com Firebase Test Lab via wrapper.

Appium 2 — quando ainda usar

Não morreu. Casos onde Appium 2 ainda faz sentido:

- **Apps híbridos legados** (WebView complexo)
- **Integração com Selenium Grid existente**
- **Drivers especializados** (Espresso driver, XCUITest driver, Mac driver)
- **Plugins customizados** (image recognition, AI-based locators)
- **Empresas com investimento Appium grande** (custo de migração)

| "Appium é o jeep antigo confiável. Maestro é o carro novo veloz." Cada caso pede um.

Appium 2 — flow equivalente

```
import { remote } from 'webdriverio';

const driver = await remote({
  capabilities: {
    platformName: 'Android',
    'appium:automationName': 'UiAutomator2',
    'appium:app': '/path/to/app.apk',
  },
});

await driver.$('~email-field').setValue('user@email.com');
await driver.$('~password-field').setValue('secret');
await driver.$('~submit-button').click();

const welcome = await driver.$('~welcome-text');
expect(await welcome.isDisplayed()).toBe(true);

await driver.deleteSession();
```

Funciona, mas verboso e dependente de WebDriver session.

Comparativo lado a lado

Mesmo flow login + asserção welcome:

Aspecto	Maestro YAML	Appium 2 TS
Linhas de código	~10	~30
Tempo de execução	~3s	~7–12s
Flake rate inicial	< 5%	~15–25%
Setup time	10min	1–2h
Curva de aprendizado	Plana	Íngreme

Numbers from real comparisons. Não é hype.

Cloud Devices — opções

Plataforma	Free tier	Forte em
Firestore Test Lab	100min/mês virtual + 5h/mês físico	Apps Android, integração Google
BrowserStack App Live	30min trial	Web + mobile, UI manual
BrowserStack App Automate	Pago	Automação cross-platform escala
LambdaTest	30min trial	Alternativa BrowserStack mais barata
Sauce Labs	Pago	Enterprise, integração CI rica
AWS Device Farm	1000min device-min free	Ecossistema AWS

Comece com Firestore Test Lab. Free tier real.

Estratégia top devices (StatCounter)

Em vez de testar tudo, **cobrir top 10–15** que somam 80%+ da base:

Brasil 2026 (StatCounter aproximado):

1. Galaxy A15 — 8%
2. Moto G54 — 6%
3. iPhone 13 — 5%
4. Galaxy A55 — 5%
5. iPhone 14 — 4%
6. Redmi Note 13 — 4%
7. Galaxy S23 — 3%
- ... etc

Variar **versões de OS** (minSdk, atual, beta) é tão importante quanto modelos.

Firebase Test Lab — exemplo

```
# Instalar gcloud
gcloud auth login
gcloud config set project meu-projeto

# Listar models disponíveis no catálogo Test Lab
gcloud firebase test android models list

# Submeter teste Android (model IDs do catálogo, não nomes AVD)
gcloud firebase test android run \
  --type instrumentation \
  --app app/build/outputs/apk/debug/app-debug.apk \
  --test app/build/outputs/apk/androidTest/debug/app-debug-androidTest.apk \
  --device model=shiba,version=34,locale=pt_BR \
  --device model=redfin,version=30,locale=pt_BR \
  --timeout 5m
```

Reports em <https://console.firebase.google.com/>. Models reais: shiba (Pixel 8), redfin (Pixel 5), oriole (Pixel 6) etc.

Flakiness mobile — causas específicas

Causa	Mitigação
Animações	<code>waitForAnimationToEnd</code> (Maestro), Idle Resources (Espresso)
OS popups (permissões)	Granting via flow ou config
Biometria	Mock em test config
Network jitter	Mock server local
Lifecycle (background→foreground)	Estabilizar antes de assert
Touch latency emulator	Rodar em real device pra confirmar

| iDFlakies (Lam et al. 2019) catalogou causas — referência acadêmica importante.

App alvo da disciplina: TestesQAMobile

```
appId: com.apptestesmobile
```

- 🍏 App Store BR: `apps.apple.com/br/app/testes-qa-mobile/id6755933674`
- 🤖 Play Store: `play.google.com/store/apps/details?id=com.apptestesmobile`
- 35 exercícios em 12 categorias com bugs propositalis

v1.1+ tem **testID** em todos elementos críticos. Convenção: `screen-elemento-acao` .

Exemplos de testIDs:

- `userform-name-input`
- `userform-submit-button`
- `calculator-digit-7`, `calculator-operator-plus`, `calculator-equals`
- `todolist-add-button`, `todolist-item-${id}`
- `home-category-functional`

Flows do exercício — scaffold

Em `puc-iec-testes-aplicacoes-mobile/exercicios/03-maestro-e2e/pratica/flows/`:

- `01-launch.yaml` — ☒ **resolvido** (modelo: `launchApp` + `assertVisible`)
- `02-userform.yaml` — ☐ preencher formulário e submeter
- `03-calculator.yaml` — ☐ `7 + 3 =` → verificar `10`
- `04-todolist.yaml` — ☐ adicionar item e ver na lista
- `05-onboarding.yaml` — ☐ completar onboarding

Bônus: `_fragments/open-home.yaml` — fragmento reusável via `runFlow:`.

Instalação Maestro completa em `docs/INSTALACA0_MAESTR0.md` (Mac/Linux/Windows + Docker).

tap0n: text vs tap0n: id — pratica estável

✗ Frágil:

```
- tap0n: "Cadastrar"
```

Quebra em: tradução do app, A/B test, refactor.

✓ Estável:

```
- tap0n:  
  id: "userform-submit-button"
```

Independente de texto. App TestesQAMobile v1.1+ suporta.

Critério 6 da Atividade 3: flows que usam id: ganham +1pt extra.

Hands-on — vamos fazer juntos (90min)

Roteiro

1. Clone o exercício: `exercicios/03-maestro-e2e/pratica` + `maestro --version` (5min)
2. Rodar `01-launch.yaml` (modelo) verde no simulador (10min)
3. Escrever `02 – 05` (userform, calculator, todolist, onboarding) (25min)
4. Extrair fragmento (`_fragments/`) + comparar 1 flow com Appium (15min)
5. Submeter 1 flow para Firebase Test Lab (15min)
6. Análise de relatório (10min)
7. Discussão (10min)

Pitfalls comuns

- ✗ `tapOn` por texto que muda — flake em A/B test ou tradução
- ✗ Não usar `runFlow` para login — repetição em todos os testes
- ✗ Suíte sem retry — flake transitório vira falha
- ✗ Executar só em emulator — esconde issues de touch latency
- ✗ Não monitorar flake rate — métrica essencial degrada silenciosamente

Atividade 3: Suíte Maestro Cross-Platform (15 pts)

Entrega: até 28/06.

Entregar:

1. **5 flows Maestro** funcionando em iOS Simulator
2. **5 flows Maestro** funcionando em Android Emulator
3. Pelo menos **1 flow em device real** via Firebase Test Lab
4. **Flakiness** < 5% em 3 runs consecutivos
5. **Matriz comparativa Maestro vs Appium** (linhas de código, tempo, taxa de sucesso) — 2pgs

Leituras obrigatórias (pré-Aula 5)

- **Maestro Docs** (oficial, mobile.dev) — completas
- **Mobile.dev Blog** — *Why YAML for E2E Tests*
- **Linares-Vásquez et al. (2017)** — *Continuous, Evolutionary and Large-Scale*. ICSME
- **OWASP MASVS v2.1** — preparação Aula 5

Próxima aula (29/06)

Aula 5 — Performance, Segurança Mobile e Observabilidade

Pré-requisito:

- Atividade 3 entregue
- Android Studio Profiler funcionando
- Xcode Instruments funcionando
- MobSF (Docker) configurado

Obrigado!

 jackson.96@gmail.com

 [linkedin.com/in/3jacksonsmith](https://www.linkedin.com/in/3jacksonsmith)

 github.com/jacksonsmith

Próxima segunda, 29/06, 19:00